# ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

## PART VIII KUBERNETES SECURITY

Zero Trust, SPIFFE, Vault, Policy-as-Code, Confidential Computing, Compliance

Volume 8 of 16 Core Series

Prerequisites: Parts I through VII

Edition 2025-2026

# TABLE OF CONTENTS

CHAPTER 1

# Enterprise Security Model for Kubernetes

Kubernetes security requires a layered, defence-in-depth approach. No single control is sufficient; attackers who defeat one layer must face additional barriers. Enterprise Kubernetes security addresses four fundamental questions: Who are you? (identity), What can you do? (authorisation), What are you running? (workload security), and What is it doing? (runtime detection).

## Kubernetes Threat Model

| Threat | Attack Vector | Kubernetes Impact | Primary Controls |
|---|---|---|---|
| Container escape | Kernel exploit, misconfiguration | Host root access, lateral movement | gVisor/Kata, seccomp, capabilities, PSS |
| Supply chain attack | Compromised image or dependency | Malicious code in production | Image signing, scanning, admission control |
| Credential theft | Stolen kubeconfig, SA token | Full cluster compromise | OIDC, short-lived tokens, RBAC audit |
| Privilege escalation | Exploit RBAC misconfiguration | Cluster-admin from low-priv access | Least privilege RBAC, no wildcards |
| Data exfiltration | Network egress from compromised Pod | Data theft | NetworkPolicy, egress control, DLP |
| Denial of Service | Resource exhaustion | Node/cluster unavailability | ResourceQuota, LimitRange, PDB |
| API server attack | Exposed API server, brute force | Cluster takeover | Private API server, auth webhook, audit |
| etcd compromise | Direct etcd access | All secrets/config exposed | TLS client auth, encryption at rest, firewall |
| Insider threat | Malicious admin | Any cluster action | Audit logging, MFA, break-glass procedures |

## Security Control Layers

```
LAYER 1: SUPPLY CHAIN Image signing (Cosign), SBOM, vulnerability scanning, SLSA provenance
LAYER 2: CLUSTER HARDENING API server flags, etcd encryption, audit logging, CIS benchmark
compliance LAYER 3: IDENTITY AND ACCESS OIDC authentication, RBAC least-privilege, SPIFFE
workload identity LAYER 4: WORKLOAD SECURITY Pod Security Standards, seccomp, AppArmor,
non-root, read-only filesystem LAYER 5: NETWORK SECURITY NetworkPolicy default-deny, mTLS
(Istio/Cilium), egress control LAYER 6: SECRET MANAGEMENT Vault/External Secrets, etcd
encryption at rest, no Secrets in Git LAYER 7: RUNTIME DETECTION Falco, Tetragon, anomaly
detection, SIEM integration LAYER 8: AUDIT AND COMPLIANCE API audit logs, compliance scanning,
```

```
penetration testing
```

CHAPTER 2

# Zero Trust Architecture on Kubernetes

Zero Trust rejects the perimeter security model. In Kubernetes, Zero Trust means: every workload must prove its identity before accessing any resource; every connection is authenticated and encrypted (mTLS); every access is authorised per-request; and every action is logged and monitored. The implicit trust within a flat cluster network is eliminated.

## Zero Trust Principles Applied to Kubernetes

* **Verify explicitly**: Every service-to-service call carries a cryptographic identity (SPIFFE SVID). The receiving service verifies the identity before processing. Implemented via service mesh mTLS (Istio, Linkerd, Cilium mTLS).
* **Least privilege access**: No workload has access beyond what it needs for its function. RBAC policies deny by default. Service accounts have minimal permissions. NetworkPolicies deny all traffic not explicitly allowed.
* **Assume breach**: Security controls assume an attacker already has code execution inside the cluster. Runtime monitoring (Falco) detects lateral movement. Blast radius is minimised through namespace isolation and network segmentation.
* **Continuous verification**: Certificates expire (short TTL). Tokens are rotated. Policy is continuously enforced at admission and runtime. Audit logs capture every action for forensic analysis.

## Zero Trust Implementation Roadmap

| Phase | Controls | Timeline | Complexity |
|---|---|---|---|
| Phase 1: Identity | OIDC SSO, service accounts with IRSA/Workload Identity, audit logging | Month 1-2 | Low |
| Phase 2: Authorisation | RBAC audit and hardening, OPA/Kyverno policies, admission webhooks | Month 2-3 | Medium |
| Phase 3: Workload | Pod Security Standards Restricted, seccomp, image signing | Month 3-4 | Medium |
| Phase 4: Network | NetworkPolicy default-deny, egress control, FQDN policies | Month 4-5 | Medium |
| Phase 5: Encryption | mTLS via Istio/Linkerd/Cilium, secrets encryption at rest | Month 5-7 | High |
| Phase 6: Runtime | Falco deployment, alert routing, SIEM integration, IR playbooks | Month 7-9 | High |
| Phase 7: Confidential | Confidential Containers, attestation, HSM integration | Month 9-12 | Very High |

CHAPTER 3

# Authentication: Users, OIDC, Service Accounts

Kubernetes supports multiple authentication mechanisms. Choosing the right mechanism for each identity type -- human users, CI/CD systems, and workloads -- is foundational to a secure cluster.

## Authentication Methods

| Method | Identity Type | Token Lifetime | Enterprise Recommendation |
|---|---|---|---|
| X.509 client certificate | Admin users (break-glass only) | Certificate validity (often years) | Avoid for regular use; use for emergency admin only |
| OIDC (OpenID Connect) | Human users, CI/CD | Short (minutes to hours) | Preferred for all human access; integrates with SSO |
| Service Account Token (static) | In-cluster workloads | Infinite (until deleted) | Legacy; avoid; use bound tokens instead |
| Service Account Token (bound) | In-cluster workloads | Configurable (1h default) | Use TokenRequest API; auto-rotated by kubelet |
| IRSA (EKS) / Workload Identity (GKE) | Cloud API access from Pods | Short (hours) | Best for cloud service access from Pods |
| Webhook Token Auth | Any | Depends on webhook | Integrates with external IdP when OIDC not available |

## OIDC Integration -- kube-apiserver Configuration

```
# kube-apiserver flags for OIDC (kubeadm ClusterConfiguration): apiServer: extraArgs:
oidc-issuer-url: https://accounts.google.com # Or corporate IdP: # oidc-issuer-url:
https://idp.company.com/realms/kubernetes oidc-client-id: kubernetes oidc-username-claim:
email oidc-groups-claim: groups oidc-username-prefix: 'oidc:' oidc-groups-prefix: 'oidc:' #
kubeconfig for OIDC user (managed by kubelogin): users: - name: my-user user: exec:
apiVersion: client.authentication.k8s.io/v1 command: kubectl args: [oidc-login, get-token,
--oidc-issuer-url=https://idp.company.com/realms/kubernetes, --oidc-client-id=kubernetes,
--oidc-client-secret=CLIENT_SECRET] # Bind OIDC group to ClusterRole: apiVersion:
rbac.authorization.k8s.io/v1 kind: ClusterRoleBinding metadata: name: platform-admins
subjects: - kind: Group name: 'oidc:platform-admins' apiGroup: rbac.authorization.k8s.io
roleRef: kind: ClusterRole name: cluster-admin apiGroup: rbac.authorization.k8s.io
```

CHAPTER 4

# RBAC: Design and Hardening

RBAC is the most frequently misconfigured Kubernetes security control. Overly permissive RBAC -- wildcard verbs, cluster-admin for application service accounts, namespace-admin for everyone -- is the most common path to cluster compromise in production environments.

## Common RBAC Roles Architecture

```
# Tiered RBAC architecture for enterprise: CLUSTER LEVEL (ClusterRole + ClusterRoleBinding):
cluster-admin: Platform team break-glass only; MFA required cluster-viewer: Read-only across
all namespaces (auditors, monitoring) node-admin: Node management for SRE team storage-admin:
StorageClass and PV management NAMESPACE LEVEL (Role + RoleBinding per namespace):
namespace-admin: Team tech lead; full access within namespace developer: Deploy, read logs,
port-forward; no secret reads readonly: View all resources; no exec or port-forward ci-cd:
Deploy only; cannot read secrets or exec SERVICE ACCOUNT LEVEL (minimal per application):
app-sa: Only what the specific application needs Example: leader-election lease read/write
only
```

## RBAC Hardening Rules

* **Never use wildcards**: spec.rules[].verbs: ['*'] or resources: ['*'] grants excessive permission. Always enumerate specific verbs and resources.
* **Never bind cluster-admin to service accounts**: Application SA with cluster-admin is a critical risk. If a container is compromised, the attacker has full cluster access.
* **Audit regularly**: Use rbac-lookup, rakkess, or kubectl-who-can to audit who can do what. Automate weekly audits.
* **Separate read and write**: Create separate roles for read and write operations. CI/CD pipelines that only read should not have write access.
* **Use aggregated ClusterRoles**: Define view/edit/admin ClusterRoles using aggregationRule; extend built-in roles cleanly.
* **Avoid ClusterRoleBindings for namespace work**: Prefer RoleBinding with ClusterRole to limit scope to a specific namespace.
* **Time-limited elevated access**: Use tools like Pinniped or custom webhooks to grant temporary elevated access with automatic expiry for break-glass scenarios.

### RBAC Audit Queries

```
# Find all subjects with cluster-admin: kubectl get clusterrolebindings -o json | \ python3 -c
" import json,sys d=json.load(sys.stdin) for item in d['items']: if item['roleRef']['name'] ==
'cluster-admin': for s in item.get('subjects',[]): print(f'{s["kind"]}: {s.get("name","?")} in
{s.get("namespace","cluster-wide")}'" # Check what a service account can do: kubectl auth
can-i --list \ --as=system:serviceaccount:production:myapp-sa \ -n production # Find roles
that allow secret access: kubectl get roles,clusterroles -A -o json | \ python3 -c " import
json,sys d=json.load(sys.stdin) for item in d['items']: for rule in item.get('rules',[]): if
'secrets' in rule.get('resources',[]): print(item['metadata']['name'])" # Install rakkess for
detailed matrix view: kubectl krew install rakkess kubectl rakkess --sa production:myapp-sa
```

CHAPTER 5

# SPIFFE and SPIRE: Workload Identity

SPIFFE (Secure Production Identity Framework For Everyone) is a CNCF graduated standard for workload identity. It assigns cryptographic identities to workloads regardless of where they run, enabling strong mutual authentication between services without managing application-level credentials.

## SPIFFE Architecture

```
SPIFFE Identity: spiffe://trust-domain/path Example:
spiffe://company.com/ns/production/sa/api-server SPIFFE Verifiable Identity Document (SVID):
X.509-SVID: X.509 certificate with SPIFFE URI in SAN JWT-SVID: JWT token with sub claim =
SPIFFE ID SPIRE (SPIFFE Runtime Environment) components: SPIRE Server (control plane): -
Maintains trust domain and registration entries - Issues SVIDs to attested workloads - CA:
self-signed or upstream (Vault PKI, AWS PCA) SPIRE Agent (DaemonSet on every node): - Attests
node identity (AWS instance identity doc, k8s PSATtest) - Delivers SVIDs to workloads via UNIX
socket - Rotates SVIDs automatically before expiry Workload API (UNIX socket:
/run/spire/sockets/agent.sock): - Workload fetches its SVID without any application changes -
No secrets, no passwords, no API keys required - SDK available for Go, Java, Python, Rust, C
```

### SPIRE on Kubernetes

```
# Install SPIRE with Helm: helm repo add spiffe https://spiffe.github.io/helm-charts/ helm
install spire spiffe/spire \ --namespace spire-system --create-namespace \ --set
global.spiffe.trustDomain=company.com \ --set spire-server.ca.subject.country=US \ --set
spire-server.ca.subject.organization=Company \ --set spire-server.ca.keyType=ec-p384 #
Register a workload: kubectl exec -n spire-system spire-server-0 -- \ spire-server entry
create \ -spiffeID spiffe://company.com/ns/production/sa/api-server \ -parentID
spiffe://company.com/k8s-workload-registrar/node \ -selector k8s:ns:production \ -selector
k8s:sa:api-server # Workload accesses SVID via Workload API: # The SPIRE agent socket is
projected into the Pod: volumes: - name: spire-agent-socket hostPath: path: /run/spire/sockets
type: DirectoryOrCreate containers: - volumeMounts: - name: spire-agent-socket mountPath:
/run/spire/sockets readOnly: true env: - name: SPIFFE_ENDPOINT_SOCKET value:
unix:///run/spire/sockets/agent.sock
```

CHAPTER 6

# Secret Management: Vault and External Secrets

Kubernetes Secrets are insufficient for enterprise secret management: they are base64-encoded (not encrypted) by default, difficult to audit, and lack dynamic secret generation, automatic rotation, and fine-grained access policies. HashiCorp Vault and the External Secrets Operator address these gaps.

## HashiCorp Vault Architecture

```
Vault provides a unified secrets management platform: Secret Engines: KV v2: Static key-value secrets with versioning PKI: Dynamic TLS certificate generation Database: Dynamic database credentials (auto-rotated) AWS/GCP/Azure: Dynamic cloud credentials Transit: Encryption-as-a-service (encrypt/decrypt data) TOTP: Time-based one-time passwords Auth Methods: Kubernetes: Pod can authenticate using SA token OIDC: Human users via SSO AWS IAM: EC2/ECS workloads LDAP: Enterprise directory Policies (HCL): Fine-grained path-based access control capabilities: [read, write, create, delete, list] # Vault on Kubernetes (HA): helm repo add hashicorp https://helm.releases.hashicorp.com helm install vault hashicorp/vault \ --namespace vault --create-namespace \ --set server.ha.enabled=true \ --set server.ha.replicas=3 \ --set server.ha.raft.enabled=true \ --set injector.enabled=true \ --set csi.enabled=true
```

## Vault Kubernetes Auth and Dynamic Database Secrets

```
# Configure Vault Kubernetes auth: vault auth enable kubernetes vault write auth/kubernetes/config \ kubernetes_host=https://kubernetes.default.svc \ kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt # Create policy for an application: vault policy write myapp-policy - <<'EOF' path "secret/data/production/myapp/*" { capabilities = ["read"] } path "database/creds/myapp-role" { capabilities = ["read"] } EOF # Create Kubernetes auth role: vault write auth/kubernetes/role/myapp \ bound_service_account_names=myapp-sa \ bound_service_account_namespaces=production \ policies=myapp-policy \ ttl=1h # Configure dynamic PostgreSQL credentials: vault secrets enable database vault write database/config/myapp-db \ plugin_name=postgresql-database-plugin \ connection_url='postgresql://vaultadmin:PASS@postgres:5432/mydb' \ allowed_roles=myapp-role vault write database/roles/myapp-role \ db_name=myapp-db \ creation_statements="CREATE ROLE ..." \ default_ttl=1h max_ttl=24h # Result: each Pod request gets a unique DB user that auto-expires
```

## External Secrets Operator

```
apiVersion: external-secrets.io/v1beta1 kind: ClusterSecretStore metadata: name: vault-backend spec: provider: vault: server: https://vault.internal.corp:8200 path: secret version: v2 caBundle: BASE64_CA_CERT auth: kubernetes: mountPath: kubernetes role: external-secrets serviceAccountRef: name: external-secrets-sa --- apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: myapp-secrets namespace: production spec: refreshInterval: 1h secretStoreRef: kind: ClusterSecretStore name: vault-backend target: name: myapp-secrets creationPolicy: Owner data: - secretKey: api-key remoteRef: key: secret/production/myapp property: api-key - secretKey: db-password remoteRef: key: secret/production/myapp property: db-password
```

CHAPTER 7

# Certificate Lifecycle: cert-manager

cert-manager is the de facto standard for TLS certificate lifecycle management in Kubernetes. It automates certificate issuance, renewal, and rotation from multiple certificate authorities: Let's Encrypt, HashiCorp Vault, AWS ACM, Venafi, and self-signed/internal CAs.

## cert-manager Architecture

```
cert-manager components: cert-manager controller: Watches Certificate resources; calls issuers cert-manager webhook: Validates and mutates cert-manager resources cert-manager cainjector: Injects CA bundles into webhook configurations Issuers: Issuer: Namespace-scoped; issues certs for one namespace ClusterIssuer: Cluster-scoped; issues certs for any namespace Issuer types: ACME (Let's Encrypt): HTTP-01 or DNS-01 challenge Vault: Vault PKI secret engine CA: Sign with a Kubernetes Secret containing CA key/cert Self-signed: Self-signed certificates (for internal CAs) Venafi: Enterprise PKI integration AWS ACM PCA: AWS Private Certificate Authority
```

### cert-manager ClusterIssuer and Certificate

```
# Internal CA ClusterIssuer (for mTLS between services): apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: internal-ca spec: ca: secretName: internal-ca-key-pair # Secret with tls.crt and tls.key --- # Vault PKI ClusterIssuer: apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: vault-issuer spec: vault: server: https://vault.internal.corp:8200 path: pki/sign/kubernetes-role caBundle: BASE64_VAULT_CA auth: kubernetes: role: cert-manager mountPath: /v1/auth/kubernetes secretRef: name: cert-manager-vault-token key: token --- # Certificate resource (auto-renewed 30 days before expiry): apiVersion: cert-manager.io/v1 kind: Certificate metadata: name: api-server-tls namespace: production spec: secretName: api-server-tls-secret duration: 2160h # 90 days renewBefore: 720h # Renew 30 days before expiry subject: organizations: [Company] commonName: api.production.svc.cluster.local dnsNames: - api.production.svc.cluster.local - api.example.com ipAddresses: - 10.96.50.100 issuerRef: name: vault-issuer kind: ClusterIssuer privateKey: algorithm: ECDSA size: 384
```

CHAPTER 8

# mTLS and Encryption in Transit

Mutual TLS (mTLS) provides both encryption and mutual authentication: both client and server present certificates and verify each other's identity. In Kubernetes, mTLS is implemented at different layers with different trade-offs.

## mTLS Implementation Options

| Approach | Where Implemented | Operation | Overhead | Best For |
|---|---|---|---|---|
| Istio sidecar mTLS | Envoy sidecar proxy | Automatic; app unaware | Medium (sidecar CPU) | Full service mesh |
| Linkerd mTLS | Rust micro-proxy | Automatic; app unaware | Low | Simple mTLS |
| Cilium mTLS | eBPF + WireGuard | Transparent L3 encryption | Low | Network-level encryption |
| SPIFFE/SPIRE + app SDK | Application code | App manages SVID | Highest control | Zero trust native apps |
| cert-manager + app | Application code | Manual cert loading | App manages rotation | Legacy apps with TLS support |

## Istio PeerAuthentication -- Strict mTLS

```
# Enforce strict mTLS for all services in a namespace: apiVersion: security.istio.io/v1 kind: PeerAuthentication metadata: name: default namespace: production spec: mtls: mode: STRICT # STRICT: all connections must use mTLS # PERMISSIVE: accept both mTLS and plaintext # DISABLE: plaintext only # Enforce mTLS cluster-wide: apiVersion: security.istio.io/v1 kind: PeerAuthentication metadata: name: default namespace: istio-system # Root namespace = cluster-wide spec: mtls: mode: STRICT # Verify mTLS is working: istioctl authn tls-check . ..svc.cluster.local # Should show: OK / mTLS
```

CHAPTER 9

# Pod Security: Standards, Contexts, Profiles

Hardening Pod security reduces the blast radius if a container is compromised. Multiple controls work together: Pod Security Standards (namespace-level policy), securityContext (per-Pod/container), seccomp (syscall filtering), and AppArmor/SELinux (mandatory access control).

## Pod Security Standards (PSS)

| Control | Privileged | Baseline | Restricted |
|---|---|---|---|
| Privileged containers | Allowed | Blocked | Blocked |
| Host namespaces (PID,Net,IPC) | Allowed | Blocked | Blocked |
| HostPath volumes | Allowed | Blocked | Blocked |
| runAsRoot | Allowed | Allowed | Must be False |
| Capabilities | Allowed | Only safe subset | Drop ALL; no adds except NET_BIND_SERVICE |
| seccompProfile | Not required | Not required | RuntimeDefault or Localhost required |
| allowPrivilegeEscalation | Allowed | Allowed | Must be False |
| readOnlyRootFilesystem | Not required | Not required | Required |
| Volume types | Any | Limited set | ConfigMap, Secret, PVC, projected, emptyDir |

### Complete Restricted Pod Security Context

```
# Fully compliant with PSS Restricted: apiVersion: v1 kind: Pod spec: securityContext: runAsNonRoot: true runAsUser: 10001 runAsGroup: 10001 fsGroup: 10001 fsGroupChangePolicy: OnRootMismatch seccompProfile: type: RuntimeDefault containers: - name: app securityContext: allowPrivilegeEscalation: false readOnlyRootFilesystem: true runAsNonRoot: true capabilities: drop: [ALL] seccompProfile: type: RuntimeDefault # Writable paths via emptyDir: volumeMounts: - { name: tmp, mountPath: /tmp } - { name: cache, mountPath: /app/cache } volumes: - { name: tmp, emptyDir: { medium: Memory } } - { name: cache, emptyDir: {} }
```

CHAPTER 10

# Policy-as-Code: OPA Gatekeeper and Kyverno

Policy-as-code enforces security and operational standards at admission time, preventing non-compliant workloads from being deployed rather than detecting violations after the fact. Two systems dominate the Kubernetes policy landscape: OPA Gatekeeper (using Rego) and Kyverno (using YAML/CEL).

## OPA Gatekeeper -- Rego Policies

```
# Gatekeeper ConstraintTemplate (defines the policy): apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate metadata: name: requiredlabels spec: crd: spec: names: kind:
RequiredLabels validation: openAPIV3Schema: type: object properties: labels: type: array
items: { type: string } targets: - target: admission.k8s.gatekeeper.sh rego: | package
requiredlabels violation[{"msg": msg}] { required := input.parameters.labels[_] not
input.review.object.metadata.labels[required] msg := sprintf("Missing required label: %v",
[required]) } --- # Constraint (applies the template): apiVersion:
constraints.gatekeeper.sh/v1beta1 kind: RequiredLabels metadata: name: require-team-label
spec: enforcementAction: deny match: kinds: [{apiGroups: [apps], kinds: [Deployment]}]
parameters: labels: [team, cost-center, environment]
```

## Kyverno -- YAML-Native Policies

```
# Kyverno policy: require non-root, read-only filesystem: apiVersion: kyverno.io/v1 kind:
ClusterPolicy metadata: name: require-pod-security-hardening spec: validationFailureAction:
Enforce background: true rules: - name: require-run-as-non-root match: resources: { kinds:
[Pod] } validate: message: Pods must not run as root pattern: spec: securityContext:
runAsNonRoot: true - name: require-readonly-rootfs match: resources: { kinds: [Pod] }
validate: message: Root filesystem must be read-only pattern: spec: containers: -
securityContext: readOnlyRootFilesystem: true --- # Kyverno generate: auto-create
NetworkPolicy on new namespace: apiVersion: kyverno.io/v1 kind: ClusterPolicy metadata: name:
add-default-networkpolicy spec: rules: - name: default-deny match: resources: { kinds:
[Namespace] } generate: apiVersion: networking.k8s.io/v1 kind: NetworkPolicy name:
default-deny-all namespace: '{{request.object.metadata.name}}' data: spec: podSelector: {}
policyTypes: [Ingress, Egress]
```

CHAPTER 11

# Supply Chain Security at Scale

At enterprise scale, supply chain security must be automated and enforced as policy rather than relying on developer discipline. The goal is a continuous verification pipeline from source code to running container.

## End-to-End Supply Chain Pipeline

```
SOURCE CODE Signed commits (GPG/SSH) Branch protection + CODEOWNERS Dependency scanning
(Dependabot, Renovate) | BUILD (CI/CD -- GitHub Actions / Tekton) BuildKit hermetic build SBOM
generation (Syft -> CycloneDX JSON) SLSA provenance generation (slsa-github-generator) Cosign
keyless signing Trivy vulnerability scan (fail on CRITICAL) Attestation upload (SBOM,
provenance, scan results) | REGISTRY (Harbor) Immutable tags enabled Policy: block CRITICAL
CVE images Continuous rescan (daily Trivy) Signature verification on push | ADMISSION
(Kubernetes) Kyverno verifyImages: signature required Kyverno: SBOM attestation required
Kyverno: only harbor.internal.corp images OPA: no known CRITICAL CVEs | RUNTIME Falco: detect
unexpected processes Tetragon: block unexpected network calls Regular re-evaluation
(continuous compliance)
```

CHAPTER 12

# Runtime Security: Falco and Tetragon

## Falco -- CNCF Runtime Security

Falco monitors system calls and Kubernetes audit events to detect anomalous behaviour at runtime. It uses eBPF probes (preferred) or a kernel module to observe every syscall made by every process in every container on a node.

```
# Critical Falco rules for AI platform security: # 1. Detect crypto mining (GPU abuse): - rule: Crypto mining process detected condition: spawned_process and container and proc.name in (xmrig, cryptonight, minerd) output: Crypto miner detected (proc=%proc.name container=%container.name) priority: CRITICAL # 2. Model exfiltration attempt: - rule: Large data transfer from inference container condition: outbound and container and k8s.pod.label.app = llm-inference and fd.net.bytes.out > 1073741824 # 1GB and not fd.rip in (10.0.0.0/8, 172.16.0.0/12) output: Possible model exfiltration (dest=%fd.rip bytes=%fd.net.bytes.out) priority: CRITICAL # 3. Prompt injection via shell escape: - rule: Shell spawned in AI workload condition: spawned_process and container and proc.name in (bash, sh, zsh) and k8s.pod.label.tier = ai output: Shell in AI container (user=%user.name cmd=%proc.cmdline) priority: WARNING
```

### Falco Alerting Integration

```
# Falco Sidekick routes alerts to multiple destinations: helm install falco falcosecurity/falco \ --set driver.kind=ebpf \ --set falcosidekick.enabled=true \ --set falcosidekick.config.slack.webhookurl=SLACK_URL \ --set falcosidekick.config.pagerduty.routingKey=PD_KEY \ --set falcosidekick.config.elasticsearch.hostport=http://elastic:9200 \ --set falcosidekick.config.alertmanager.hostport=http://alertmanager:9093 # Falco metrics for Prometheus: falco_events_total{rule=~'.*',priority='CRITICAL'} > 0 # Alert: any CRITICAL Falco event in production namespace
```

CHAPTER 13

# Confidential Containers and Confidential Computing

Confidential Computing protects data in use -- while it is being processed in memory -- using hardware-based Trusted Execution Environments (TEEs). This addresses the attack vector that all other security controls leave open: a privileged attacker (cloud provider admin, compromised hypervisor) reading memory.

## Confidential Computing Hardware

| Technology | Vendor | Protection | Kubernetes Integration |
|---|---|---|---|
| Intel TDX (Trust Domain Extensions) | Intel (4th gen Xeon+) | VM-level memory encryption; hardware attestation | Kata Containers + TDX; COCO project |
| AMD SEV-SNP (Secure Encrypted Virtualisation) | AMD (EPYC 3rd gen+) | VM memory encrypted; SNP = nested page integrity | Kata Containers + SEV-SNP; widely available |
| ARM CCA (Confidential Compute Architecture) | ARM (v9.2+) | Realm VMs; hardware attestation | Emerging; ARM server adoption growing |
| Intel SGX (Software Guard Extensions) | Intel (older) | Enclave-level (user app) | EGo, Gramine; complex; limited memory |
| NVIDIA H100 Confidential Computing | NVIDIA | GPU memory encryption; TEE attestation | NVIDIA Confidential Containers; AI workloads |

## Confidential Containers (CoCo) Project

The Confidential Containers CNCF project enables running standard OCI containers inside hardware TEEs with cryptographic attestation. This provides confidentiality for AI model weights and inference data against cloud provider-level attackers.

```
CoCo architecture: Kubernetes API | RuntimeClass: kata-remote (Confidential Containers runtime) | cloud-api-adaptor (connects Kubernetes to cloud TEE API) | TEE Virtual Machine (AMD SEV-SNP / Intel TDX) Encrypted memory; hardware attestation | Attestation Agent (in VM) Provides evidence to Attestation Service | Attestation Service (KBS - Key Broker Service) Verifies hardware attestation Releases secrets only to attested workloads # Deploy with CoCo RuntimeClass: apiVersion: node.k8s.io/v1 kind: RuntimeClass metadata: name: kata-remote handler: kata-remote --- # Pod using confidential computing: spec: runtimeClassName: kata-remote containers: - name: llm-inference image: company/llm-server:v1 # Model weights decrypted only inside TEE # Cloud provider cannot access model or inference data
```

CHAPTER 14

# Kubernetes Security Hardening Checklist

## Control Plane

* API server: --anonymous-auth=false
* API server: --audit-log-path, --audit-log-maxage=30, --audit-log-maxbackup=3
* API server: --tls-min-version=VersionTLS12
* API server: --disable-admission-plugins does not include NodeRestriction
* etcd: --client-cert-auth=true, --peer-client-cert-auth=true
* etcd: encryption at rest enabled (EncryptionConfiguration with AES-GCM or KMS)
* Scheduler: --profiling=false
* Controller manager: --profiling=false, --use-service-account-credentials=true

## Node/kubelet

* kubelet: --anonymous-auth=false, --authorization-mode=Webhook
* kubelet: --rotate-certificates=true, --rotate-server-certificates=true
* kubelet: --protect-kernel-defaults=true
* kubelet: --event-qps=0 (disable event rate limiting for audit)
* Node OS: CIS hardened image (Bottlerocket, Flatcar, Ubuntu CIS)
* Node OS: SSH access restricted; no direct SSH in production (use kubectl debug)
* Node: no direct internet access; egress via NAT gateway or proxy

## Workloads

* Pod Security Standards: Restricted for all production namespaces
* All Pods: runAsNonRoot: true, readOnlyRootFilesystem: true
* All Pods: capabilities drop: ALL
* All Pods: seccompProfile: RuntimeDefault
* No Pods with hostNetwork, hostPID, hostIPC
* No privileged containers
* Resource requests and limits on all containers

## RBAC and Identity

* No cluster-admin bindings for service accounts
* No wildcard verbs or resources in RBAC rules
* ServiceAccount automountServiceAccountToken: false by default
* OIDC for human authentication (not static kubeconfig with certs)
* Workload Identity (IRSA/GKE WI) for cloud API access

## Network

* Default-deny NetworkPolicy in all production namespaces
* API server not publicly exposed (private endpoint only)
* Node-to-node communication encrypted (Cilium WireGuard or IPsec)
* Egress NetworkPolicy restricts outbound to known endpoints

CHAPTER 15

# Compliance Mapping: NIST, CIS, SOC2, PCI, HIPAA, EU AI Act

Enterprise Kubernetes deployments must satisfy multiple overlapping compliance frameworks. The following mapping shows which Kubernetes controls satisfy requirements from the most common frameworks.

| Kubernetes Control | NIST SP 800-53 | CIS K8s Benchmark | SOC2 | PCI DSS | HIPAA | EU AI Act |
|---|---|---|---|---|---|---|
| RBAC least privilege | AC-6 | 5.1 | CC6.3 | 7.1.2 | 164.312(a) | Art.9 (access) |
| Audit logging | AU-2, AU-12 | 3.2 | CC7.2 | 10.2 | 164.312(b) | Art.13 (transparency) |
| etcd encryption at rest | SC-28 | 1.2.33 | CC6.7 | 3.5 | 164.312(a)(2)(iv) | Art.10 (data) |
| mTLS / encryption in transit | SC-8 | 5.7 | CC6.7 | 4.1 | 164.312(e)(2)(ii) | Art.10 |
| Image signing + SBOM | SA-12, CM-14 | N/A | CC7.1 | 6.3.2 | 164.312(c) | Art.15 (supply chain) |
| Pod Security Standards | SI-7 | 5.2 | CC6.1 | 2.2 | 164.312(c) | Art.9 |
| Network policy default-deny | SC-7 | 5.3 | CC6.6 | 1.2 | 164.312(e) | Art.9 |
| Secret management (Vault) | IA-5, SC-28 | Implicit | CC6.7 | 8.3 | 164.312(a) | Art.10 |
| Runtime detection (Falco) | SI-4, IR-5 | Implicit | CC7.3 | 10.6 | 164.312(b) | Art.14 (monitoring) |
| Vulnerability scanning | RA-5, SI-2 | Implicit | CC7.1 | 6.3.3 | 164.308(a)(1) | Art.15 |
| Backup + DR | CP-9, CP-10 | Implicit | A1.2 | 12.10.1 | 164.308(a)(7) | Art.12 |

## EU AI Act -- Kubernetes-Specific Requirements

The EU AI Act (fully applicable August 2026) introduces specific requirements for high-risk AI systems that directly impact Kubernetes AI platform design:

* **Article 9 -- Risk management**: Implement RBAC, NetworkPolicy, and admission controls to enforce access boundaries around AI model access and inference endpoints.
* **Article 10 -- Data governance**: Use encrypted PVCs for training data, implement data lineage tracking (MLflow), enforce namespace isolation between data categories.
* **Article 12 -- Record keeping**: Maintain immutable audit logs (Kubernetes audit + Falco events) stored outside the cluster. Log all model access, inference requests to high-risk models, and configuration changes.
* **Article 13 -- Transparency**: Maintain SBOM for all AI components. Document model versions, training data provenance in model registry.
* **Article 15 -- Accuracy and robustness**: Implement canary deployments and A/B testing for model updates. Monitor inference accuracy metrics in Prometheus/Grafana.

CHAPTER 16

# Security Incident Response on Kubernetes

When a Kubernetes security incident occurs -- compromised container, credential theft, suspicious lateral movement -- the response must be fast and systematic. Kubernetes provides unique tools for containment and forensics.

## Incident Response Playbook

```
PHASE 1: DETECTION (seconds to minutes) Falco alert fires: shell spawned in production container Alert routed to: PagerDuty -> on-call engineer Correlate with: Kubernetes audit logs, network flows (Hubble) PHASE 2: CONTAINMENT (minutes) # Immediately isolate compromised Pod with NetworkPolicy: kubectl label pod COMPROMISED_POD quarantine=true kubectl apply -f - <<'YAML' apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: {name: quarantine} spec: podSelector: matchLabels: {quarantine: 'true'} policyTypes: [Ingress, Egress] # No rules = deny all traffic YAML # Scale down compromised Deployment (remove from Service): kubectl scale deployment COMPROMISED_DEPLOY --replicas=0 PHASE 3: FORENSICS (minutes to hours) # Capture Pod state before deletion: kubectl describe pod COMPROMISED_POD > forensics/pod-describe.txt kubectl get events --field-selector involvedObject.name=COMPROMISED_POD kubectl logs COMPROMISED_POD --all-containers > forensics/pod-logs.txt kubectl exec -it COMPROMISED_POD -- ps aux > forensics/processes.txt # Export filesystem for analysis: crictl export CONTAINER_ID forensics/container-fs.tar # Review API audit log: grep COMPROMISED_POD /var/log/kubernetes/audit.log | jq . PHASE 4: REMEDIATION Rotate any credentials the compromised container had access to Rebuild affected images from scratch Apply missing security controls (PSS, NetworkPolicy, seccomp) Update Falco rules if new attack pattern identified
```

CHAPTER 17

# Hands-On Exercises

## Exercise 8.1 -- RBAC Hardening

Audit and fix over-privileged RBAC:

```
# Identify over-privileged service accounts: kubectl auth can-i --list \
--as=system:serviceaccount:default:default -n default # Create minimal service account:
kubectl create serviceaccount myapp-minimal-sa -n default # Create minimal Role (read own
configmap only): kubectl create role myapp-role \ --verb=get --resource=configmaps \
--resource-name=myapp-config -n default kubectl create rolebinding myapp-binding \
--role=myapp-role \ --serviceaccount=default:myapp-minimal-sa -n default # Verify: can read
own configmap: kubectl auth can-i get configmaps/myapp-config \
--as=system:serviceaccount:default:myapp-minimal-sa -n default # Verify: cannot read secrets:
kubectl auth can-i get secrets \ --as=system:serviceaccount:default:myapp-minimal-sa -n
default
```

## Exercise 8.2 -- Kyverno Policy Enforcement

Deploy Kyverno and enforce security policies:

```
# Install Kyverno: helm repo add kyverno https://kyverno.github.io/kyverno/ helm install
kyverno kyverno/kyverno --namespace kyverno --create-namespace # Apply policy requiring
non-root: kubectl apply -f - <<'YAML' apiVersion: kyverno.io/v1 kind: ClusterPolicy metadata:
name: require-nonroot spec: validationFailureAction: Enforce rules: - name: nonroot match:
resources: { kinds: [Pod] } validate: message: Must run as non-root pattern: spec:
securityContext: runAsNonRoot: true YAML # Try to deploy a root container (should be blocked):
kubectl run root-test --image=nginx --restart=Never # Expected: admission webhook denied the
request # Deploy compliant Pod: kubectl run nonroot-test --image=nginx:alpine \
--overrides='{"spec":{"securityContext":{"runAsNonRoot":true,"runAsUser":101}}}'
```

### End of Part VIII -- Continue to Part IX: Platform Engineering

Part IX covers the complete platform engineering stack: GitOps with ArgoCD and Flux, Helm and Kustomize patterns, Backstage Internal Developer Platform, Crossplane for infrastructure-as-code, progressive delivery with Argo Rollouts, multi-tenancy architectures (vCluster, namespace-as-a-service), cluster lifecycle management, and platform engineering for AI workloads.